DESENVOLVIMENTO MOBILE · MÓDULO 13 · TRILHA KOTLIN · NÍVEL INTERMEDIÁRIO

Recursos do Dispositivo e IoT

Permissões, câmera e QR code, localização, notificações, WorkManager, sensores, Bluetooth Low Energy e MQTT com ESP32

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Solicitar permissões em tempo de execução seguindo as boas práticas.
- Usar câmera, seletor de fotos, leitor de QR code e localização.
- Criar notificações e agendar tarefas em segundo plano com WorkManager.
- Ler sensores do aparelho como `Flow`.
- Comunicar com um **ESP32** via **Bluetooth Low Energy** (GATT).
- Receber telemetria via **MQTT** e escolher o protocolo adequado.

Atenção

Este módulo exige **celular físico**: o emulador não tem Bluetooth real.

Roteiro

Neste módulo

- Fundamentos: hardware, sensores e comunicação
- Permissões em tempo de execução
- Câmera, QR code e localização
- Notificações e segundo plano
- Sensores do aparelho
- Bluetooth Low Energy com ESP32
- MQTT e escolha do protocolo
- Fechamento

Fundamentos: hardware, sensores e comunicação

Do mundo físico ao número



Conversor analógico-digital (ADC)

Mede a tensão em instantes regulares (**taxa de amostragem**) e a representa com um número de **bits**. Um ADC de 12 bits, como o do ESP32, distingue $2^{12} = 4096$ níveis.

Teorema da amostragem

Para reconstruir um sinal, amostre a mais que o dobro da maior frequência presente nele (Nyquist). Abaixo disso surgem frequências falsas (*aliasing*).

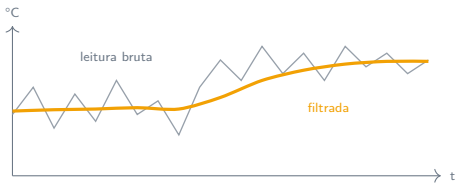
Sensores MEMS

Estruturas mecânicas microscópicas gravadas no silício. O **acelerômetro** mede aceleração (inclusive a da gravidade); o **giroscópio**, velocidade angular; o **magnetômetro**, o campo magnético; o **barômetro**, a pressão.

Atenção

Toda medida tem ruído, deriva e erro de calibração: exiba com a precisão que o sensor realmente tem.

Filtrando e combinando sensores



Média móvel exponencial

$y \leftarrow y + \alpha\,(x - y)$, com α entre 0 e 1. Pouco código, pouca memória: ótima para suavizar leituras.

Passa-baixas e passa-altas

Um filtro **passa-baixas** remove variações rápidas (ruído); um **passa-altas** remove variações lentas (deriva, gravidade). Quanto menor o α, mais suave e mais atrasada fica a curva.

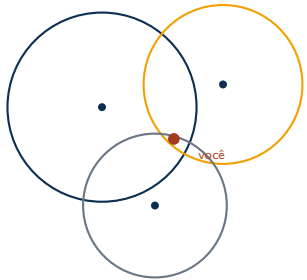
Fusão de sensores

Combinar sensores para compensar os defeitos de cada um. O giroscópio é preciso no curto prazo e deriva; o acelerômetro é ruidoso, mas não deriva. Juntos, dão a orientação do aparelho.

Dica

O sistema já entrega sensores fundidos, como o vetor de rotação. Prefira-os a reinventar a fusão.

Como o GPS descobre onde você está



cada satélite define uma esfera de distância; a interseção é a posição

GNSS e trilateração

Satélites transmitem a hora de seus relógios atômicos; o receptor calcula a distância pelo tempo de viagem do sinal. Com **quatro** satélites, resolve latitude, longitude, altitude e o erro do próprio relógio.

Várias constelações e ajuda da rede

Celulares usam GPS (EUA), GLONASS (Rússia), Galileo (UE) e BeiDou (China). O A-GPS baixa pela internet a posição dos satélites e reduz o primeiro acerto de minutos para segundos.

Atenção

Dentro de prédios o sinal some e o sistema recorre a Wi-Fi, torres e sensores. Ao ar livre, a precisão típica é de alguns metros.

Permissões e privacidade

Tipo	Concessão	Exemplos
Normal	automática, declarada no manifesto	internet, vibração
Em tempo de execução	o usuário decide num diálogo	câmera, localização, Bluetooth, notificações
Especial	tela própria nas configurações	alarmes exatos, sobrepor apps

Um pouco de história

O Android pede permissões em tempo de execução desde a versão 6.0 (2015). Vieram depois a permissão de uma só vez (11), a localização aproximada (12) e o seletor de fotos sem permissão (13).

Menor privilégio e minimização

Peça só o que o recurso exige, só quando ele for usado, e colete só os dados necessários. É boa engenharia e é exigência da LGPD (Módulo 15).

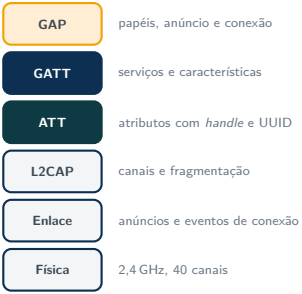
No iOS

Cada permissão precisa de um texto de justificativa no `Info.plist`; sem ele, o app é encerrado ao pedir o recurso.

Dica

Um ponto colorido na barra de status avisa quando câmera ou microfone estão em uso, no Android e no iOS.

Bluetooth Low Energy por dentro



Um pouco de história

O BLE surgiu no Bluetooth 4.0 (2010) para sensores alimentados por baterias pequenas. O Bluetooth 5 (2016) trouxe o dobro de velocidade e um modo de longo alcance.

Por que gasta tão pouco

O rádio fica quase sempre desligado. O periférico anuncia em 3 dos 40 canais e, conectado, acorda só a cada **intervalo de conexão** (de 7,5 ms a 4 s) para trocar alguns bytes.

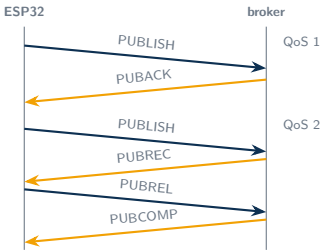
MTU

Tamanho máximo de um pacote ATT: 23 bytes por padrão (20 de dados úteis). Central e periférico podem negociar até 517 bytes.

Dica

Leituras frequentes: use **notify**. Mais dados por segundo: aumente o MTU e reduza o intervalo de conexão, sabendo que isso custa bateria.

MQTT por dentro



Um pouco de história

Andy Stanford-Clark (IBM) e Arlen Nipper criaram o MQTT em 1999 para monitorar oleodutos via satélite. Virou padrão OASIS em 2014; a versão 5.0 é de 2019.

Leve de propósito

Roda sobre TCP (porta 1883, ou 8883 com TLS) e o cabeçalho fixo tem só 2 bytes. Mensagens de *keep-alive* (PINGREQ e PINGRESP) mostram ao broker que a placa continua viva.

Sessão persistente

Com sessão persistente, o broker guarda as assinaturas e as mensagens de QoS 1 e 2 enquanto o cliente está desconectado e as entrega na volta.

Comunicação serial e UART



UART

Comunicação serial **assíncrona**: não há fio de relógio. Os dois lados combinam a velocidade (*baud rate*, como 115 200) e o formato, quase sempre **8N1**: 8 bits de dados, sem paridade, 1 bit de parada.

Ligação

TX de um lado no RX do outro, e terra comum. Atenção aos níveis: o ESP32 trabalha em 3,3 V, o Arduino Uno em 5 V e o RS-232 em ± 12 V.

Do USB à serial

A placa aparece no celular como dispositivo USB; um conversor (CH340, CP2102, FTDI) ou o próprio microcontrolador traduz para UART.

Monte um protocolo

A serial só entrega bytes. Defina mensagens com delimitador (como `\n`) e, se o ambiente for ruidoso, uma soma de verificação.

NFC: comunicação por proximidade

NFC

Comunicação a 13,56 MHz por acoplamento indutivo, com alcance de poucos centímetros. A etiqueta passiva não tem bateria: é alimentada pelo campo do próprio leitor.

Modo	Uso
Leitura e escrita	etiquetas com URL, texto ou identificador
Emulação de cartão	o celular se passa por cartão (pagamentos, crachás)

NDEF

Formato padrão das mensagens gravadas em etiquetas: uma lista de registros, cada um com tipo (URI, texto, MIME) e conteúdo.

No LabMonitor

Uma etiqueta colada na bancada guarda `https://labmonitor.app/bancada/b3`: encostar o celular abre a tela da bancada por link profundo (Módulo 07).

Atenção

A distância curta é uma proteção, mas não substitui a autenticação: qualquer um pode ler ou copiar uma etiqueta sem proteção.

Energia: o recurso mais escasso

Consumidor	Por que gasta	Como economizar
Tela	maior consumo do aparelho	tema escuro em telas OLED; nada de manter a tela ligada sem motivo
GPS	receptor ligado e cálculos contínuos	localização aproximada, intervalos maiores, cercas virtuais
Rede móvel	rádio de alta potência e tempo de cauda	agrupar requisições, *push* em vez de *polling*
CPU	despertares frequentes	agendadores do sistema em vez de temporizadores próprios
Bluetooth	varreduras longas	filtros por serviço e varreduras curtas

Um pouco de história

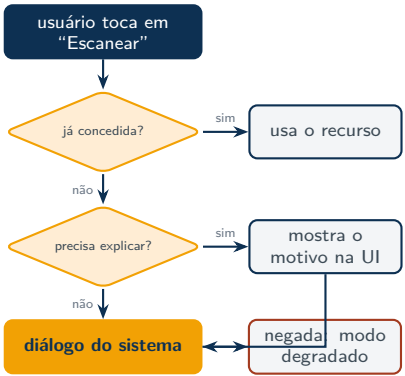
O Android criou o modo Doze (6.0, 2015), limitou serviços em segundo plano (8.0, 2017) e passou a classificar apps por frequência de uso (9.0, 2018). O iOS sempre restringiu o segundo plano a modos específicos.

Regra geral

Deixe o sistema decidir **quando** executar: WorkManager no Android, `BGTaskScheduler` no iOS. Ele agrupa o trabalho de vários apps e acorda o aparelho menos vezes.

Permissões em tempo de execução

Fluxo de permissões



Boas práticas

- Peça **no momento do uso**, não na abertura do app.
- Explique o motivo **antes** quando não for óbvio.
- Peça o **mínimo**: localização aproximada se bastar.
- Funcione, mesmo que limitado, se o usuário negar.

Atenção

Após duas negativas, o sistema não mostra mais o diálogo. Ofereça um botão que abre as configurações do app.

Pedindo permissões em Compose

PermissaoBle.kt

```
val permissoesBle = if (Build.VERSION.SDK_INT >= 31)
  arrayOf(Manifest.permission.BLUETOOTH_SCAN,
          Manifest.permission.BLUETOOTH_CONNECT)
else arrayOf(Manifest.permission.ACCESS_FINE_LOCATION)

@Composable
fun BotaoProcurarSensores(onPermitido: () -> Unit) {
  var negado by remember { mutableStateOf(false) }
  val launcher = rememberLauncherForActivityResult(
    ActivityResultContracts.RequestMultiplePermissions()
  ) { resultado ->
    if (resultado.values.all { it }) onPermitido()
    else negado = true
  }
  Button(onClick = { launcher.launch(permissoesBle) }) {
    Text("Procurar sensores")
  }
  if (negado) Text("Sem Bluetooth não é possível " +
    "ler as bancadas. Ative em Configurações.")
}
```

AndroidManifest.xml

```
<uses-permission android:name=
 "android.permission.BLUETOOTH_SCAN"
 android:usesPermissionFlags=
  "neverForLocation" />
<uses-permission android:name=
 "android.permission.BLUETOOTH_CONNECT"/>
<!-- Android 11 ou anterior -->
<uses-permission android:name=
 "android.permission.
   ACCESS_FINE_LOCATION"
 android:maxSdkVersion="30" />
```

Dica

Declare no manifesto **e** peça em tempo de execução. `neverForLocation` afirma que o app não usa o BLE para localizar o usuário.

Câmera, QR code e localização

Fotos e QR code sem permissão

Seletor de fotos (Photo Picker)

```
val escolher = rememberLauncherForActivityResult(
  ActivityResultContracts.PickVisualMedia()
) { uri -> uri?.let(vm::onFotoEscolhida) }

Button(onClick = {
  escolher.launch(PickVisualMediaRequest(
    ActivityResultContracts.PickVisualMedia.ImageOnly))
}) { Text("Escolher foto") }
```

Leitor de QR code (Google Play services)

```
suspend fun lerPatrimonio(ctx: Context): String? =
  GmsBarcodeScanning.getClient(ctx)
    .startScan().await().rawValue  // ex.: "IFMA-0042"
```

Menos permissões, mais privacidade

Photo Picker e **leitor de códigos** rodam no sistema: o app recebe só o resultado, sem permissão de câmera ou galeria.

Ideia para o laboratório

QR code com o patrimônio em cada equipamento: ler o código abre a ficha no LabMonitor.

Dica

Para preview de câmera personalizado, análise quadro a quadro ou visão computacional, use **CameraX** (exige a permissão `CAMERA`).

Localização

LocalizacaoRepository.kt

```
class LocalizacaoRepository @Inject constructor(
  @ApplicationContext ctx: Context
) {
  private val cliente =
    LocationServices.getFusedLocationProviderClient(ctx)

  @RequiresPermission(anyOf = [
    Manifest.permission.ACCESS_COARSE_LOCATION,
    Manifest.permission.ACCESS_FINE_LOCATION])
  suspend fun atual(): Location? =
    cliente.getCurrentLocation(
      Priority.PRIORITY_BALANCED_POWER_ACCURACY,
      null
    ).await()
}
```

Permissão	Precisão
`COARSE`	aproximada (cerca de 3 km)
`FINE`	precisa (GPS)
`BACKGROUND`	também com o app fechado

Fused Location Provider

Combina GPS, Wi-Fi e rede celular, escolhendo a fonte mais econômica para a precisão pedida.

Atenção

Localização em segundo plano passa por revisão rigorosa da Google Play. Só peça se for essencial.

Notificações e segundo plano

Notificações locais

Notificador.kt

```
const val CANAL_ALERTAS = "alertas"

fun criarCanal(ctx: Context) {
  val canal = NotificationChannel(CANAL_ALERTAS,
    "Alertas de temperatura",
    NotificationManager.IMPORTANCE_HIGH)
  ctx.getSystemService(NotificationManager::class.java)
    .createNotificationChannel(canal)
}

fun notificarAlerta(ctx: Context, bancada: Int, t: Double) {
  val n = NotificationCompat.Builder(ctx, CANAL_ALERTAS)
    .setSmallIcon(R.drawable.ic_termometro)
    .setContentTitle("Bancada $bancada: %.1f °C".format(t))
    .setContentText("Acima do limite configurado")
    .setAutoCancel(true)
    .build()
  if (temPermissaoNotificacao(ctx))
    NotificationManagerCompat.from(ctx).notify(bancada, n)
}
```

Canais

Desde o Android 8, toda notificação pertence a um **canal**. O usuário controla som, vibração e bloqueio de cada canal separadamente.

Android 13+

É preciso pedir `POST_NOTIFICATIONS` em tempo de execução, como qualquer permissão perigosa.

Dica

Adicione um `PendingIntent` com `setContentIntent` para abrir a tela da bancada ao tocar na notificação.

WorkManager: tarefas garantidas

SincronizarWorker.kt

```
@HiltWorker
class SincronizarWorker @AssistedInject constructor(
  @Assisted ctx: Context, @Assisted p: WorkerParameters,
  private val repo: BancadaRepository
) : CoroutineWorker(ctx, p) {
  override suspend fun doWork(): Result = try {
    repo.enviarPendentes(); repo.sincronizar()
    Result.success()
  } catch (e: IOException) { Result.retry() }
}

val pedido = PeriodicWorkRequestBuilder<SincronizarWorker>(
    15, TimeUnit.MINUTES)                    // mínimo: 15 min
  .setConstraints(Constraints(
    requiredNetworkType = NetworkType.CONNECTED,
    requiresBatteryNotLow = true))
  .build()
WorkManager.getInstance(ctx).enqueueUniquePeriodicWork(
  "sync", ExistingPeriodicWorkPolicy.KEEP, pedido)
```

Quando usar

Trabalho que **precisa acontecer** mesmo se o app fechar ou o aparelho reiniciar: sincronizar, enviar a fila offline, limpar o banco.

Necessidade	Ferramenta
enquanto a tela está aberta	coroutines
adiável e garantida	WorkManager
contínua e visível (ex.: monitorar BLE)	*foreground service*
horário exato (alarme)	`AlarmManager`

Sensores do aparelho

Sensores como `Flow`

Usando o callbackFlow do Módulo 06

```
class InclinacaoViewModel @Inject constructor(
  sensores: SensorManager
) : ViewModel() {
  val anguloX: StateFlow<Float> = sensores
    .leituras(Sensor.TYPE_ACCELEROMETER)
    .map { (x, y, z) ->
      Math.toDegrees(atan2(x, hypot(y, z)).toDouble())
        .toFloat()
    }
    .sample(100)       // no máximo 10 Hz para a UI
    .stateIn(viewModelScope,
      SharingStarted.WhileSubscribed(5_000), 0f)
}
```

Sensor	Medida
Acelerômetro	aceleração em m/s^2 (x, y, z)
Giroscópio	velocidade angular em rad/s
Magnetômetro	campo magnético em μT
Luz	iluminância em lux
Barômetro	pressão em hPa
Proximidade	distância (ou perto/longe)

Celular como instrumento

Com o magnetômetro dá para detectar o campo de um ímã ou de um indutor; com o acelerômetro, medir vibração de motores (Fourier com `Dispatchers.Default`).

Bluetooth Low Energy com ESP32

Conceitos de BLE



- **GAP**: anúncio (*advertising*) e conexão. O ESP32 anuncia; o celular procura e conecta.
- **GATT**: organização dos dados em **serviços** e **características**, cada uma com um UUID.
- Operações: **read**, **write** e **notify** (o periférico envia quando o valor muda).

Dica

Use o app **nRF Connect** para explorar e depurar o ESP32 antes de escrever o código Android.

Firmware do ESP32 (Arduino)

labmonitor_ble.ino -- configuração

```
#include <BLEDevice.h>   // inclui BLEServer.h
#include <BLE2902.h>
#define SVC_UUID  "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define TEMP_UUID "beb5483e-36e1-4688-b7f5-ea07361b26a8"
BLECharacteristic *temp;
void setup() {
  BLEDevice::init("LabMonitor-B3");
  BLEServer *srv = BLEDevice::createServer();
  BLEService *svc = srv->createService(SVC_UUID);
  temp = svc->createCharacteristic(TEMP_UUID,
    BLECharacteristic::PROPERTY_READ |
    BLECharacteristic::PROPERTY_NOTIFY);
  temp->addDescriptor(new BLE2902());   // CCCD
  svc->start();
  BLEDevice::getAdvertising()->addServiceUUID(SVC_UUID);
  BLEDevice::startAdvertising();
}
```

-- laço principal

```
void loop() {
  // centésimos de grau: 24,87 °C -> 2487
  int16_t c = round(lerTemperatura() * 100);
  temp->setValue((uint8_t*)&c, sizeof(c));
  temp->notify();
  delay(1000);
}
```

Formato binário

`int16_t` em centésimos de grau: 2 bytes, *little-endian*.

Contrato

UUIDs, escala e ordem dos bytes: documente o contrato e gere UUIDs próprios.

Procurando o ESP32 no Android

BleScanner.kt

```
val SVC_UUID = UUID.fromString(
  "4fafc201-1fb5-459e-8fcc-c5c9c331914b")

@SuppressLint("MissingPermission") // verificada antes
fun BluetoothAdapter.procurar(): Flow<ScanResult> =
  callbackFlow {
    val cb = object : ScanCallback() {
      override fun onScanResult(tipo: Int, r: ScanResult) {
        trySend(r)
      }
      override fun onScanFailed(erro: Int) {
        close(IllegalStateException("scan falhou: $erro"))
      }
    }
    val filtro = ScanFilter.Builder()
      .setServiceUuid(ParcelUuid(SVC_UUID)).build()
    val cfg = ScanSettings.Builder()
      .setScanMode(ScanSettings.SCAN_MODE_LOW_LATENCY).build()
    bluetoothLeScanner.startScan(listOf(filtro), cfg, cb)
    awaitClose { bluetoothLeScanner.stopScan(cb) }
  }
```

Filtre pelo serviço

O filtro por UUID faz o sistema entregar só os seus dispositivos e economiza bateria.

Na UI

`procurar().take(10)` com `timeout` de 10 s preenche uma lista com nome e `rssi` (intensidade do sinal) de cada bancada.

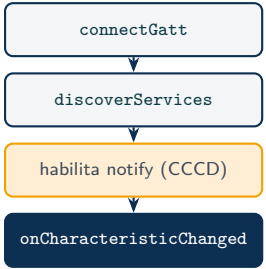
Atenção

O scan é caro: pare assim que encontrar o dispositivo. O Android limita a 5 inícios de scan a cada 30 s por app.

Conectando ao periférico

BleTemperatura.kt (1/2)

```
fun BluetoothDevice.temperaturas(ctx: Context): Flow<Double> =
  callbackFlow {
    val cb = object : BluetoothGattCallback() {
      override fun onConnectionStateChange(
          g: BluetoothGatt, status: Int, novo: Int) {
        if (novo == BluetoothProfile.STATE_CONNECTED)
          g.discoverServices()
        else close(IOException("desconectado ($status)"))
      }
      override fun onServicesDiscovered(g: BluetoothGatt, s: Int) {
        val c = g.getService(SVC_UUID).getCharacteristic(TEMP_UUID)
        g.setCharacteristicNotification(c, true)
        g.writeDescriptor(c.getDescriptor(CCCD_UUID),
          BluetoothGattDescriptor.ENABLE_NOTIFICATION_VALUE)
      }
    } // + onCharacteristicChanged (próximo slide)
    val gatt = connectGatt(ctx, false, cb,
      BluetoothDevice.TRANSPORT_LE)
    awaitClose { gatt.disconnect(); gatt.close() }
  }
```



Dica

`CCCD_UUID` é o UUID padrão `00002902-0000-1000-8000-00805f9b34fb`.

Recebendo e decodificando as notificações

BleTemperatura.kt (2/2)

```
override fun onCharacteristicChanged(
    g: BluetoothGatt,
    c: BluetoothGattCharacteristic,
    valor: ByteArray) {
  val bruto = ByteBuffer.wrap(valor)
    .order(ByteOrder.LITTLE_ENDIAN).short   // int16
  trySend(bruto / 100.0)                    // °C
}
```

No ViewModel: reconexão automática

```
val temperatura = dispositivo.temperaturas(ctx)
  .retryWhen { causa, tentativa ->
    causa is IOException && tentativa < 5 &&
      run { delay(1_000L shl tentativa.toInt()); true }
  }
  .stateIn(viewModelScope, WhileSubscribed(5_000), null)
```

Cuidados reais

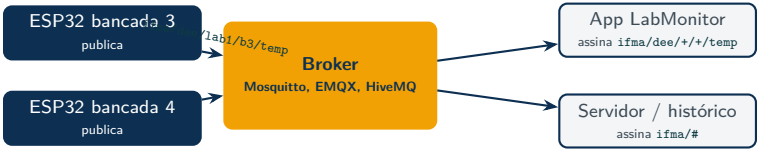
- Uma operação GATT por vez: enfileire leituras e escritas.
- Sempre `close()` ao terminar.
- Reconecte com espera crescente (1, 2, 4, 8… s).

Bibliotecas e segundo plano

Kable e **Nordic BLE Library** cuidam de fila e reconexão. Monitorar com a tela desligada exige um *foreground service*.

MQTT e escolha do protocolo

MQTT: publicar e assinar



Tópicos

Hierarquia com /. Curingas: + (um nível) e # (todos os níveis abaixo).

QoS

0: no máximo uma vez. **1**: pelo menos uma vez. **2**: exatamente uma vez.

Retained / LWT

Retained guarda o último valor do tópico. *Last Will* avisa quando a placa cai.

MQTT no app

MqttRepository.kt (HiveMQ MQTT Client)

```
fun leiturasMqtt(filtro: String): Flow<Pair<String, Double>> =
  callbackFlow {
    val cliente = MqttClient.builder()
      .useMqttVersion5()
      .identifier("app-" + UUID.randomUUID())
      .serverHost("broker.labmonitor.ifma.edu.br")
      .serverPort(8883).sslWithDefaultConfig()   // TLS
      .buildAsync()
    cliente.connectWith()
      .simpleAuth().username(usuario)
        .password(senha.toByteArray()).applySimpleAuth()
      .send().await()
    cliente.subscribeWith().topicFilter(filtro)
      .callback { msg ->
        val valor = String(msg.payloadAsBytes).toDouble()
        trySend(msg.topic.toString() to valor)
      }.send().await()
    awaitClose { cliente.disconnect() }
  }
```

ESP32 (PubSubClient)

```
mqtt.setServer(BROKER, 1883);
mqtt.connect("esp32-b3", USER, PASS,
  "ifma/dee/lab1/b3/status", 1, true,
  "offline");                    // LWT
mqtt.publish("ifma/dee/lab1/b3/temp",
  String(t, 2).c_str(), true); // retained
```

Segurança

Broker público só para testes. Em produção: TLS (porta 8883), usuário e senha ou certificados, e regras de acesso por tópico.

Dica

`await()` sobre `CompletableFuture` vem de `kotlinx-coroutines-jdk8`.

Qual protocolo usar?

	BLE	HTTP na rede local	MQTT via broker
Alcance	∼10–30 m, sem infraestrutura	a rede Wi-Fi	qualquer lugar com internet
Consumo no sensor	muito baixo (bateria)	alto (Wi-Fi)	médio (Wi-Fi, conexão persistente)
Tempo real	sim (notify)	não (*polling*)	sim (push)
Vários clientes	um central por vez (em geral)	sim	sim, muitos
Histórico na nuvem	não	não	fácil (assinante no servidor)
Complexidade no app	alta (GATT)	baixa (Retrofit)	média
Caso ideal	configurar a placa, leitura de bancada ao lado	painel na rede do laboratório	monitoramento contínuo e alertas remotos

Arquitetura combinada

Use BLE para o *provisionamento* (enviar SSID e senha do Wi-Fi ao ESP32 pelo app) e MQTT para a telemetria do dia a dia.

Fechamento

Resumo

Resumo

- Peça permissões no momento do uso, explique o motivo e funcione mesmo se negadas.
- Photo Picker e leitor de códigos dispensam permissões; CameraX e localização exigem.
- Notificações precisam de canal e, no Android 13+, de permissão; WorkManager garante tarefas adiáveis.
- Sensores, scan BLE, GATT e MQTT viram `Flow` com `callbackFlow` + `awaitClose`.
- BLE: periférico, serviços, características e CCCD; firmware e app compartilham um contrato binário.
- Escolha BLE, HTTP local ou MQTT conforme alcance, energia, tempo real e número de clientes.

Glossário (1/2)

Termo	Significado
Permissão em tempo de execução	autorização dada pelo usuário num diálogo do sistema
ADC	conversor que transforma uma tensão em número
Taxa de amostragem	quantas medidas são feitas por segundo
Sensor MEMS	sensor micromecânico gravado no silício (acelerômetro, giroscópio)
Fusão de sensores	combinação de sensores para compensar os defeitos de cada um
GNSS	sistemas de navegação por satélite: GPS, GLONASS, Galileo, BeiDou
Trilateração	cálculo da posição a partir das distâncias a pontos conhecidos
WorkManager	agenda trabalho garantido em segundo plano
Doze	modo que adia rede e alarmes com o aparelho parado

Glossário (2/2)

Termo	Significado
BLE	Bluetooth de baixo consumo, feito para sensores
GATT	organização dos dados BLE em serviços e características
Característica	valor com UUID que pode ser lido, escrito ou notificado
MTU	tamanho máximo de um pacote ATT (23 bytes por padrão)
MQTT	protocolo leve de publicação e assinatura, com *broker*
QoS	nível de garantia de entrega do MQTT: 0, 1 ou 2
UART	comunicação serial assíncrona (formato 8N1, *baud rate*)
NDEF	formato padrão das mensagens em etiquetas NFC

Questões de revisão

1. Descreva o caminho de uma temperatura do sensor do ESP32 até a tela. Onde entram o ADC e a taxa de amostragem?
2. Por que o acelerômetro sozinho não basta para saber a orientação do celular?
3. Quantos satélites o GPS precisa e por quê? Por que, dentro do laboratório, a posição piora?
4. Quando pedir uma permissão, e o que fazer se o usuário negar?
5. Por que o BLE consome tão pouco? O que é o intervalo de conexão?
6. Que QoS do MQTT usar para leituras a cada segundo e para o comando que desliga o relé? Por quê?
7. Explique o formato 8N1 e o que acontece se os dois lados usarem *baud rates* diferentes.
8. Como reduzir o consumo de bateria de um app que monitora a bancada o dia todo?

Exercícios

Exercício 1 — QR code

Gere QR codes com números de patrimônio, cole em objetos e implemente a leitura que abre a ficha do equipamento.

Exercício 2 — BLE

Com um ESP32 e um sensor (DHT22, DS18B20 ou o sensor interno), implemente o firmware GATT do slide e uma tela que lista os dispositivos encontrados e mostra a temperatura em tempo real.

Exercício 3 — Escrita GATT

Adicione uma característica *WRITE* que liga e desliga um relé (ou LED). Crie um `Switch` no app para controlá-la.

LabMonitor — Etapa 13

Troque o simulador por leituras reais: BLE para a bancada próxima e MQTT para o painel geral. Gere uma notificação quando a temperatura passar do limite e agende a sincronização com WorkManager.

Para aprofundar

- Permissões no Android: `https://developer.android.com/guide/topics/permissions/overview`
- Photo Picker: `https://developer.android.com/training/data-storage/shared/photo-picker`
- Leitor de códigos: `https://developers.google.com/ml-kit/vision/barcode-scanning/code-scanner`
- WorkManager: `https://developer.android.com/develop/background-work/background-tasks/persistent`
- Bluetooth Low Energy: `https://developer.android.com/develop/connectivity/bluetooth/ble/ble-overview`
- ESP32 Arduino Core: `https://docs.espressif.com/projects/arduino-esp32`
- HiveMQ MQTT Client: `https://github.com/hivemq/hivemq-mqtt-client` MQTT: `https://mqtt.org`

Respostas das questões de revisão

1. O sensor transforma temperatura em tensão; o ADC do ESP32 amostra e quantiza em 12 bits; o *firmware* converte em °C e envia por BLE ou MQTT; o app decodifica e exibe. A taxa de amostragem define as leituras por segundo; acima do necessário, só gasta energia.
2. Ele mede a gravidade somada à aceleração do movimento e é ruidoso: com o aparelho em movimento, a inclinação calculada erra. A fusão com o giroscópio (vetor de rotação) resolve.
3. Quatro: três coordenadas e o erro do relógio do receptor. Dentro de prédios, o sinal é bloqueado ou refletido e o sistema recorre a Wi-Fi e torres, menos precisos.
4. No momento em que o recurso é usado, explicando o motivo quando não for óbvio. Se o usuário negar, o app segue funcionando sem o recurso e oferece um atalho para as configurações.
5. O rádio fica quase sempre desligado e só acorda nos eventos de conexão. O intervalo de conexão é o tempo entre esses encontros, de 7,5 ms a 4 s.
6. Leituras: QoS 0 ou 1, pois uma perdida logo é substituída pela seguinte. Relé: QoS 1 com comando idempotente (ou QoS 2), pois perder o comando é grave e repeti-lo não pode causar dano.
7. Um bit de início, 8 de dados, nenhum de paridade e 1 de parada. Com *baud rates* diferentes, o receptor amostra nos instantes errados e lê lixo.
8. MQTT ou *push* em vez de *polling*, envios agrupados, taxa de amostragem menor, *notify* no BLE, WorkManager para tarefas periódicas e nada de GPS contínuo.

Fim do Módulo 13

Próximo: Módulo 14 — Testes e qualidade: testes unitários e de UI, corotinas em testes, desempenho e acessibilidade.